Spécification d'Ingénierie Système et Architecture de Rupture pour Assistant Mobile Autonome

L'automatisation et l'orchestration autonome des téléphones intelligents traversent une transformation architecturale majeure. Historiquement cantonnés à des échanges textuels épisodiques ou à des assistants vocaux rigides fondés sur des grammaires fermées, les assistants mobiles évoluent vers des agents neuro-symboliques capables de piloter l'intégralité du système d'exploitation, d'accéder sans friction au patrimoine informationnel de l'utilisateur et d'anticiper proactivement ses besoins.
Cette spécification d'ingénierie définit les fondations matérielles, logicielles et algorithmiques nécessaires à la conception d'un assistant de rupture sur smartphone. Elle intègre les avancées les plus récentes issues de la recherche en systèmes agentiques, en vision par ordinateur pour interfaces graphiques, en compression de modèles pour accélérateurs neuronaux et en protocoles de contexte unifié.

1. Thèse Architecturale : Le Paradigme Neuro-Symbolique Hybride

Les tentatives d'automatisation mobile se heurtent traditionnellement à deux écueils opposés. D'un côté, l'approche programmatique pure — reposant sur des scripts rigides et l'analyse de fichiers d'arbres d'accessibilité XML — échoue dès qu'une interface devient dynamique, s'affiche dans un canevas graphique ou subit une mise à jour visuelle mineure. De l'autre côté, l'approche purement visuelle — où un modèle vision-langage généraliste analyse chaque capture d'écran brute — engendre une latence prohibitive, une surconsommation de bande passante mémoire et une saturation rapide de la fenêtre de contexte.
Pour surmonter ces verrous, l'architecture de référence adopte un **paradigme neuro-symbolique hybride à double voie d'exécution** :

1. **La voie symbolique directe (OS-Native)** : L'assistant privilégie systématiquement l'invocation d'interfaces de programmation typées et déclaratives fournies par le système d'exploitation (telles qu'AppFunctions sous Android et App Intents sous iOS) ainsi que l'interrogation de bases de données personnelles via le protocole unifié Model Context Protocol (MCP). Cette voie garantit une exécution déterministe en quelques millisecondes et une consommation énergétique minimale.
2. **La voie neuronale réflexive (Visual GUI Fallback)** : Lorsque les applications cibles n'exposent aucune interface déclarative ou que des flux imprévus surviennent, l'assistant bascule de manière transparente sur un pipeline visuel multimodal. Ce dernier combine la segmentation d'écran haute résolution, l'ancrage spatial par marquage explicite et une topologie multi-agents orchestrant la planification, la décision tactile et la vérification post-action.

Ce découpage permet de concilier la robustesse d'un contrôle bas niveau avec la flexibilité cognitive nécessaire pour interagir avec n'importe quel écosystème applicatif tiers.

2. Substrats d'Intégration OS et Primitives d'Exécution Matérielle

L'exécution concrète d'actions et la manipulation de données sur un appareil mobile exigent un interfaçage profond avec les couches privilégiées du système d'exploitation. Les architectures modernes se structurent selon les spécificités d'Android et d'iOS.

Architecture Système Android : AppFunctions, Android MCP et Services d'Accessibilité

Sur la plateforme Android, l'intégration combine les fonctionnalités natives d'Android 16 (API Level 36) et d'Android 17 (API Level 37) avec le framework système d'accessibilité.

Invocation Déclarative via AppFunctions

La bibliothèque Jetpack androidx.appfunctions formalise l'exposition des capacités applicatives aux agents d'intelligence artificielle. Les applications déclarent leurs points d'entrée à l'aide de l'annotation @AppFunction. Lors de la phase de compilation, le compilateur KSP (Kotlin Symbol Processing) extrait les métadonnées et génère les schémas structurels enregistrés dans le registre système.
Chaque fonction reçoit une instance de AppFunctionContext qui renseigne l'identité de l'appelant, ses habilitations et les ressources locales. Les objets transférés à travers les frontières inter-processus doivent implémenter AppFunctionSerializable.

```
// Déclaration d'une capacité applicative native via AppFunctions
@AppFunction(
    name = "create_calendar_entry",
    description = "Ajoute un événement structuré à l'agenda de
l'utilisateur"
)
suspend fun addCalendarEvent(
    context: AppFunctionContext,
    eventData: CalendarEventPayload
): CalendarEventResult {
    // Vérification cryptographique des droits de l'assistant
appelant
    if (!context.callerMetadata.isAuthorizedAssistant) {
        throw SecurityException("Appelant non autorisé.")
    }
    return CalendarRepository.insert(eventData)
}
```

Orchestration par le Model Context Protocol (Android MCP)

Android 17 introduit l'Android Model Context Protocol au cœur de sa couche middleware. Agissant comme un bus sémantique sur l'appareil, Android MCP permet à l'agent de découvrir dynamiquement les fonctions enregistrées dans le registre AppFunctions, d'interroger le moteur d'indexation locale AppSearch et d'orchestrer des requêtes inter-applications sans exfiltrer d'état applicatif vers le cloud.

Primitives Graphiques d'Accessibilité

Lorsque la voie déclarative n'est pas supportée par une application tierce, l'assistant sollicite son implémentation de AccessibilityService. Ce service fournit les primitives fondamentales suivantes :

- **Inspection de la structure d'affichage** : Surveillance des flux d'événements via AccessibilityEvent et extraction de fenêtres actives avec getRootInActiveWindow().
- **Synthèse de gestes tactiles** : La méthode dispatchGesture() prend en argument un objet GestureDescription configuré par des tracés Path, autorisant la simulation

précise d'appuis simples, de pressions continues et de balayages multi-touch sans distinction avec un doigt humain.

- **Actions globales d'orchestration** : La fonction performGlobalAction() permet d'injecter directement plus de trente actions système, incluant GLOBAL_ACTION_BACK, GLOBAL_ACTION_HOME, GLOBAL_ACTION_RECENTS, GLOBAL_ACTION_NOTIFICATIONS et la capture d'écran.

```
// Contrôleur d'injection gestuelle et de commandes système
class HardwareGestureController(private val service:
AccessibilityService) {

    fun injectTap(x: Float, y: Float, durationMs: Long = 50L,
onComplete: () -> Unit) {
        val tapPath = Path().apply { moveTo(x, y) }
        val stroke = GestureDescription.StrokeDescription(tapPath,
0, durationMs)
        val gesture =
GestureDescription.Builder().addStroke(stroke).build()

        service.dispatchGesture(gesture, object :
AccessibilityService.GestureResultCallback() {
            override fun onCompleted(gestureDescription:
GestureDescription?) {
                onComplete()
            }
        }, null)
    }

    fun injectSwipe(startX: Float, startY: Float, endX: Float,
endY: Float, durationMs: Long = 300L) {
        val swipePath = Path().apply {
            moveTo(startX, startY)
            lineTo(endX, endY)
        }
        val stroke =
GestureDescription.StrokeDescription(swipePath, 0, durationMs)

service.dispatchGesture(GestureDescription.Builder().addStroke(str
oke).build(), null, null)
    }
}
```

Architecture Système iOS : App Intents, Entités Sécurisées et Passerelle d'Automatisation

L'écosystème Apple impose un cloisonnement applicatif strict. L'intégration s'effectue sur deux niveaux distincts selon la nature des applications cibles.

Le Framework Déclaratif App Intents

Sous iOS (étendu sur iOS 18 et versions ultérieures), les applications exposent leurs actions et états au système via le framework AppIntents. Les objets de données sont modélisés par le protocole AppEntity, tandis que la synchronisation inter-appareils (iPhone, iPad, Mac) est assurée par SyncableEntity. Les attributs sont indexés dans Spotlight via l'annotation @Property(indexingKey:).
Pour encadrer les opérations sensibles, Apple impose des protocoles de sécurité spécifiques :

- OwnershipProvidingEntity : Renvoie une valeur EntityOwnership exigeant une authentification biométrique explicite avant toute altération de données partagées.
- LongRunningIntent : Étend l'exécution en arrière-plan via performBackgroundTask.
- CancellableIntent : Intercepte les signaux IntentCancellationReason pour dissocier les annulations manuelles des temporisations système.
- UndoableIntent : Fournit des mécanismes de réversibilité d'état.

```
// Exemple de déclaration d'App Intent sous iOS
struct SendEncryptedMessageIntent: AppIntent {
    static var title: LocalizedStringResource = "Envoyer un
message chiffré"

    @Parameter(title: "Destinataire")
    var recipient: ContactEntity

    @Parameter(title: "Contenu")
    var message: String

    @MainActor
    fun perform() async throws -> some IntentResult {
        guard let contact = try? await $recipient.requestValue()
else {
            throw IntentError.contactNotFound
        }
        MessageDispatcher.send(to: contact, text: message)
        return .result()
    }
}
```

Prise de Contrôle Visuelle sous iOS

En raison de l'impossibilité pour une application tierce d'inspecter l'écran ou d'émettre des gestes globaux sans restriction, le contrôle visuel autonome sous iOS nécessite d'activer le Mode Développeur et d'associer l'appareil à une passerelle d'automatisation. L'assistant communique avec un serveur local basé sur WebDriverAgent (WDA), adossé au moteur natif XCTest. Les instructions de l'agent sont converties via le Model Context Protocol en requêtes HTTP pilotant WDA pour capturer l'écran et simuler les interactions tactiles.
Caractéristique Système,Écosystème Android,Écosystème Apple iOS
Framework Déclaratif,AppFunctions (androidx.appfunctions),App Intents Framework
Agent Système Natif,Gemini Intelligence / Assistant Core,Siri AI / Apple Intelligence

Protocole de Données Locales,Android MCP (Model Context Protocol),App Intents Registry & Spotlight Index
Contrôle Graphique Global,AccessibilityService (dispatchGesture),Passerelle WebDriverAgent (XCTest) via MCP
Extraction des Métadonnées,Kotlin Symbol Processing (KSP) à la compilation,appintentsmetadataprocessor dans Xcode
Socle Matériel Minimal,NPU compatible avec Android AICore,Apple Silicon A17 Pro ou puce M-Series

3. Perception Visuelle, Résolution Dynamique et Ancrage Spatial

Pour interagir avec des applications dépourvues d'APIs déclaratives, l'assistant doit convertir des captures d'écran brutes en représentations spatiales et sémantiques précises. Les affichages mobiles présentent des défis sévères : des ratios d'aspect étirés, une forte densité d'informations et des cibles tactiles de petite dimension (icônes, texte compact).

Traitement des Ratios Étirés : L'Architecture Multi-Résolution

La réduction brute d'une capture d'écran mobile au format d'entrée carré conventionnel d'un modèle vision-langage (par exemple 448×448 ou 512×512 pixels) dégrade drastiquement la résolution des micro-composants, provoquant des hallucinations de coordonnées spatiales.
Pour résoudre ce goulot d'étranglement, le pipeline intègre le principe de partitionnement dynamique emprunté à Ferret-UI. L'image en mode portrait ($9:16$ ou $9:20$) est découpée horizontalement en deux sous-images à haute résolution, encodées individuellement en parallèle d'une version globale compressée de l'écran complet. Les caractéristiques visuelles locales sont ensuite projetées dans l'espace de représentation textuelle du modèle, préservant la netteté des icônes et des caractères typographiques fins. En complément, l'intégration d'un mécanisme de focalisation sur zone d'intérêt (tel que proposé par Spotlight) permet à l'agent de concentrer son attention cognitive sur une boîte englobante candidate plutôt que de réencoder l'intégralité du canevas visuel lors des étapes de vérification.

Pipeline de Parsing d'Écran : OmniParser v2 et Marquage Set-of-Mark

La structuration de l'écran repose sur un pipeline séquentiel hautement spécialisé :

1. **Extraction de la Capture d'Écran Brute** : Récupération du tampon d'affichage en pleine définition (1080×2400 pixels) via les API système.
2. **Détection d'Objets UI par Modèle Spécialisé** : Un détecteur YOLOv8 optimisé pour les interfaces graphiques mobiles identifie l'ensemble des zones cliquables, champs de saisie et icônes pour générer leurs boîtes englobantes.
3. **Légendage Sémantique** : Un modèle de vision compact basé sur Florence-2 produit une description textuelle précise de la fonction associée à chaque élément détecté.
4. **Surimpression Numérique Set-of-Mark (SoM)** : Le système superpose au centre de chaque composant candidat une étiquette numérique distincte (1, 2, 3...), éliminant ainsi toute prédiction de coordonnées continues par le grand modèle de langage.
5. **Sélection Discrète de Cible** : L'agent cognitif reçoit l'image annotée et sélectionne simplement l'identifiant entier désiré (par exemple {"action": "click", "target_id": 3}).

6. **Conversion Géométrique et Injection Gestuelle** : Une table de mise en correspondance interne résout l'identifiant sélectionné en coordonnées physiques absolues (x, y) , transmises immédiatement au contrôleur dispatchGesture().

OmniParser v2 affiche une exactitude de 39,5 % sur le benchmark de positionnement ScreenSpot Pro et opère avec une latence moyenne d'environ 0,6 seconde par image sur processeur graphique dédié.

Mise à l'Échelle au Moment du Test (Visual Test-Time Scaling)

Les interfaces mobiles comportent de nombreux états visuels instables : animations de chargement, transitions d'écrans fluides ou bannières éphémères. L'assistant déploie une stratégie de mise à l'échelle au moment du test (*Visual Test-Time Scaling*).
Lorsqu'un niveau d'incertitude élevé est détecté sur un élément d'interface, le système alloue dynamiquement des ressources de calcul supplémentaires : il capture deux images successives espacées de 100 ms pour s'assurer de la stabilité visuelle, extrait des zooms recadrés sur la zone cible et compare les prédictions avant de déclencher l'action physique.

4. Topologie Cognitive Multi-Agents et Boucles de Réflexion Hiérarchiques

L'utilisation d'un modèle de langage unique en boucle fermée ReAct (Reason + Act) provoque systématiquement des échecs sur les tâches mobiles longues. La dérive de contexte et l'accumulation d'historiques visuels saturent la mémoire du modèle, entraînant des boucles infinies ou des oublis de la consigne initiale.
L'architecture résout ce défi en adoptant la topologie multi-agents tripartie introduite par Mobile-Agent-v2 et étendue par Mobile-Agent-E et MobileUse.

Découplage Fonctionnel des Rôles

- **L'Agent de Planification (Manager)** : Responsable exclusif de la vision globale et de la stratégie. Il reçoit la requête complexe de l'utilisateur, maintient l'agenda des sous-objectifs et synthétise l'avancement global sous forme d'états textuels abstraits. Le Manager ne traite jamais d'images brutes ou de listes d'éléments visuels denses, ce qui protège son contexte de tout encombrement prématuré.
- **L'Agent de Décision (Operator)** : Reçoit du Manager un sous-objectif atomique immédiat (ex. *"Cliquer sur la barre de recherche et taper 'Boulangerie'"*), la capture d'écran annotée par Set-of-Mark et la liste des composants d'OmniParser v2. Il détermine l'élément cible et produit l'instruction d'action structurée en JSON.
- **L'Agent de Réflexion (Reflector)** : Opère immédiatement après l'exécution de chaque commande. En comparant les captures d'écran avant et après l'opération, il évalue si l'état du système a changé comme prévu (ex. le menu s'est-il déroulé ? le clavier est-il apparu ?). En cas d'anomalie, il injecte un signal correctif immédiat à l'Operator ou demande au Manager de reformuler son plan.

La mise en œuvre de cette séparation fonctionnelle démontrée dans Mobile-Agent-v2 améliore le taux de succès des tâches de plus de 30 % comparativement aux agents monolithiques.

Architecture de Réflexion Multi-Échelle (Three-Tier Reflection)

Pour garantir une résilience maximale contre les imprévus d'interface, le système implémente le modèle de réflexion à trois niveaux de granularité temporelle formalisé par MobileUse :

- **Niveau 1 : Action Reflection (Échelle atomique)** : Analyse la transition d'état visuel immédiatement après chaque clic ou saisie. Il vérifie l'expansion d'un menu ou la modification d'un bouton, permettant d'intercepter instantanément les clics dans le vide.
- **Niveau 2 : Trajectory Reflection (Échelle tactique)** : Évalue la cohérence d'une séquence de 3 à 5 actions successives. Ce niveau identifie les dérives de trajectoire ou les oscillations entre deux écrans et commande une interruption pour réorienter la stratégie.
- **Niveau 3 : Global Reflection (Échelle stratégique)** : Valide la complétion totale de la tâche avant la restitution finale à l'utilisateur, en comparant l'état final de l'appareil aux contraintes et critères de succès de la consigne initiale.

5. Système de Mémoire Persistante, Auto-Évolution et Dual-RAG

Pour être véritablement intelligent, un assistant mobile ne peut repartir de zéro à chaque session. Il doit conserver une mémoire structurelle de l'utilisateur, apprendre de ses erreurs passées et optimiser ses trajectoires au fil du temps.

Hiérarchisation de la Mémoire Inspirée des Systèmes d'Exploitation (MemGPT)

L'architecture intègre le paradigme de gestion de mémoire virtuelle de MemGPT, structurant la mémoire en niveaux hiérarchiques :

- **Contexte Actif (Équivalent RAM)** : Tokens directement transmis dans la fenêtre de contexte du modèle lors de chaque inférence. Il comprend les directives système immuables (persona, sécurité), le contexte de travail dynamique (profil utilisateur, état courant) et un tampon circulaire FIFO des derniers échanges.
- **Contexte Externe (Équivalent Disque)** :
- *Stockage de Rappel (Recall Storage)* : Base d'archivage textuelle complète des sessions antérieures, accessible par recherche sémantique.
- *Stockage d'Archivage (Archival Storage)* : Base de données vectorielle et graphes de connaissances contenant les documents de référence et les préférences permanentes.

Lorsque le tampon FIFO approche des limites de contexte, le système exécute une pagination automatique : les faits essentiels sont consolidés dans le contexte de travail, tandis que les détails bruts sont expulsés vers le stockage de rappel. L'agent peut interroger sa mémoire externe à tout moment via des appels d'outils dédiés (recall_storage.search).

Graphe de Connaissance Dynamique (Mem0)

Pour relier les entités privées entre elles sans saturer le modèle de données redondantes, l'assistant déploie un moteur de mémoire fondé sur des triplets entité-relation inspiré de Mem0 :

$$\text{Sujet} \xrightarrow{\quad\text{Prédicat}\quad} \text{Objet}$$

Ce graphe de relations permet d'extraire chirurgicalement les seuls contextes pertinents pour une tâche donnée. Les benchmarks documentent que l'approche par graphe réduit de 91 % la latence au 95e percentile ($p95$) et diminue de plus de 90 % le volume de tokens consommés comparativement à la transmission intégrale des historiques de conversation.

Mécanisme d'Auto-Évolution : Conseils (Tips) et Raccourcis (Shortcuts)

Issu des principes de Mobile-Agent-E, l'assistant intègre un sous-système d'auto-évolution qui analyse chaque trajectoire de tâche terminée :

- **Génération de Conseils (*Tips*)** : Après un échec résolu ou une difficulté d'interface, l'agent extrait une règle heuristique qu'il persiste dans sa base de données (ex. *« Sur l'application SNCF Connect, la sélection d'une date requiert de valider deux fois le calendrier »*). Ces conseils sont automatiquement injectés au Manager lors des requêtes ultérieures visant la même application.
- **Génération de Raccourcis (*Shortcuts*)** : Lorsqu'une sous-tâche complexe et récurrente a été exécutée avec succès, l'agent en extrait une macro-commande paramétrée assortie de préconditions visuelles. Lors des exécutions futures, l'agent rejoue directement ce raccourci déterministe sans faire appel au modèle de vision pour chaque étape intermédiaire, divisant le temps de traitement par quatre. Mobile-Agent-E démontre une amélioration absolue de plus de 22 % sur l'efficacité globale grâce à cette capitalisation.

Architecture RAG à Double Niveau (Dual-Level RAG)

Pour guider les agents sans encombrer leur fenêtre d'attention, le système implémente le schéma de Mobile-Agent-RAG :

- **Manager-RAG** : Interroge une base de connaissances globale pour fournir au planificateur des décompositions stratégiques types pour des tâches abstraites non familières.
- **Operator-RAG** : Récupère au niveau de l'opérateur des triplets visuels spécifiques composés de l'objectif immédiat, de la capture d'écran de référence et de l'action tactile validée, permettant d'orienter le geste tactile avec une fiabilité accrue (+11,0 % de taux de complétion et +10,2 % d'efficacité par étape).

6. Moteur de Proactivité Autonome et Anticipation d'Intention

Un assistant qui n'agit que sur commande explicite reste un simple exécutant. Pour devenir un collaborateur indispensable, il doit être doté d'une capacité d'anticipation contextuelle autonome.

Modélisation Act2Intention

L'anticipation repose sur un modèle d'inférence continue *Act2Intention* . L'assistant analyse passivement les séquences d'actions et les événements ambiants pour déduire l'objectif sous-jacent avant même que l'utilisateur ne le formule :
$$\text{Flux d'Événements} \longrightarrow \text{Compréhension Contextuelle} \longrightarrow \text{Prédiction d'Intention} \longrightarrow \text{Proposition Proactive}$$
Si l'utilisateur consulte une confirmation de réservation de vol sur son application de messagerie, l'agent infère le besoin logistique connexe, vérifie l'agenda pour repérer les conflits potentiels et prépare un itinéraire vers l'aéroport avec une estimation des conditions de circulation.

Déclencheurs Systèmes d'Arrière-Plan

La proactivité est orchestrée techniquement par plusieurs services natifs Android :

- NotificationListenerService : Analyse les notifications entrantes (messages, e-mails de transport, alertes de livraison) pour détecter les événements déclencheurs sans mobiliser l'écran.
- WorkManager : Planifie des routines d'audit périodiques respectueuses de la batterie (ex. préparation du briefing matinal à 7h00).
- AppSearch : Moteur d'indexation locale haute performance qui maintient un graphe sémantique à jour des documents et messages locaux.

Exploration Visuelle à Froid (Cold-Start Exploration)

Lorsqu'une nouvelle application est installée, un agent autonome standard présente un taux d'échec élevé lors de sa première utilisation. Pour éliminer ce problème, l'architecture reprend le mécanisme d'exploration proactive de MobileUse.
Pendant les heures creuses (la nuit, lorsque le smartphone est branché sur secteur et connecté au Wi-Fi), l'agent lance des sessions d'exploration autonome : il ouvre la nouvelle application, parcourt ses écrans, clique sur les menus principaux, dresse la cartographie des composants interactifs et peuple sa base de raccourcis (*Shortcuts*) et de conseils (*Tips*). Dès la première commande de l'utilisateur, l'agent dispose déjà d'un graphe d'accessibilité complet de l'application.

7. Accélération Matérielle NPU, Compression Algorithmique et Gestion Mémoire

Pour opérer en temps réel, préserver l'autonomie de la batterie et garantir la confidentialité des données privées, l'assistant doit exécuter ses composants d'inférence et d'indexation vectorielle directement sur le matériel mobile.

Le Verrou de la Bande Passante Mémoire et l'Intensité Opérationnelle

Les communications grand public mettent fréquemment en avant les capacités théoriques des NPU en Tera Operations Per Second (TOPS) : 73 TOPS pour le Snapdragon 8 Elite, 35 TOPS pour l'Apple A18 Pro ou 40 TOPS pour le Dimensity 9400. Cependant, ces mesures brutes ne reflètent pas les contraintes réelles lors de l'exécution d'un modèle de langage sur smartphone.
L'exécution d'un modèle s'articule en deux phases microarchitecturales distinctes :

- **La phase de préremplissage (*Prefill*)** : Traite les tokens d'entrée en parallèle pour construire le cache Key-Value (KV). Cette étape est limitée par la puissance de calcul matriciel du NPU et peut atteindre des cadences de 780 tokens par seconde sur des modèles légers.
- **La phase de décodage (*Decode*)** : Génère les tokens de sortie de manière séquentielle et autorégressive, un par un. Pour chaque token produit, l'ensemble des poids synaptiques du réseau doit être transféré depuis la mémoire vive LPDDR5X vers les mémoires caches SRAM locales du processeur.

En conséquence, l'intensité opérationnelle de la phase de décodage chute à environ $1\text{ opération par octet}$ (1 op/byte). La vitesse de génération maximale est alors strictement dictée par la bande passante de la mémoire vive :
$$\text{Cadence de Décodage (tokens/s)} \approx \frac{\text{Bande Passante Mémoire (Go/s)}}{\text{Taille du Modèle en RAM (Go)}}$$
Sur un smartphone équipé d'une puce Snapdragon 8 Elite disposant d'une bande passante mémoire effective de 62 Go/s , le débit opérationnel réel du NPU lors du décodage plafonne à :

$$\text{Débit Opérationnel Effectif} = 62\text{ Go/s} \times 1\text{ op/byte} = 0{,}062\text{ TOPS}$$
Rapporté aux 73 TOPS théoriques annoncés, le taux d'utilisation effectif des unités de calcul du NPU est inférieur à $0{,}1\%$. Le processeur passe l'immense majorité de ses cycles en attente de données bloquées sur le bus système. De surcroît, la dissipation thermique passive d'un châssis de téléphone plafonne la consommation continue admissible entre 2,5 et 4,0 Watts. Réduire la taille mémoire des modèles est donc la condition absolue pour accélérer l'inférence.

Compression Algorithmique et Quantification Bas-Bit

Pour loger des modèles de plusieurs milliards de paramètres dans l'enveloppe de 2 à 4 Go allouée aux applications par le système d'exploitation, l'emploi de techniques de quantification avancées est indispensable. La quantification entière uniforme projette les poids haute précision (FP16) vers des formats entiers discrets selon la formule :
$$x_{\text{quant}} = \text{round}\left(\frac{x}{s} \right) + z$$
où s représente le facteur d'échelle flottant et z le décalage du point zéro.
L'architecture déploie les formats de compression suivants :

- **INT4 / FP4 avec AWQ (Activation-aware Weight Quantization)** : Réduit l'empreinte mémoire d'un facteur 4 sans dégradation mesurable du raisonnement en protégeant les canaux de poids associés aux pointes d'activation critiques. Un modèle de 3 milliards de paramètres (ex. Llama 3.2 3B, Phi-4-mini ou Gemma 2 2B) n'occupe plus que $1{,}5\text{ Go}$ en RAM et soutient plus de 45 tokens par seconde.
- **Précision Mixte (Mixed-Precision)** : Les matrices d'attention sensibles (projections W_q, W_k, W_v) et les couches d'embeddings sont maintenues en INT8 ou FP16, tandis que les couches Feed-Forward Network (FFN), plus volumineuses et moins sensibles, sont quantifiées en INT4 ou FP4.
- **Architecture Ternaire BitNet b1.58** : Les poids sont restreints à l'ensemble discret $\{-1, 0, 1\}$. Les multiplications matricielles énergivores (MAC) sont totalement remplacées par de simples additions entières au niveau du silicium, réduisant drastiquement l'échauffement thermique par token généré.

Pipeline RAG Embarqué et Gestion Mémoire NPU

Pour indexer et interroger les données locales de l'utilisateur, l'infrastructure déploie un pipeline RAG localisé :

- **Segmentation de Texte Adaptée** : Les documents locaux sont découpés en fenêtres de 1 000 caractères avec un chevauchement de 500 caractères, respectant la taille des graphes mémoires statiques alloués par les moteurs NPU.
- **Embeddings Locaux** : Un modèle ultra-léger (ex. EmbeddingGemma 300M) génère des vecteurs de dimension 384 ou 768 directement sur le NPU en moins de 15 ms.
- **Moteur Vectoriel Embarqué** : Utilisation conjointe de sqlite-vec (via des tables virtuelles vec0) pour l'intégration SQL directe et d'ObjectBox pour les graphes HNSW (Hierarchical Navigable Small World) avec accès mémoire sans copie (*zero-copy*), assurant des recherches de similarité en moins de 30 ms.
- **Règle d'Ordonnancement Mémoire NPU** : Sur des architectures comme Qualcomm Hexagon, les modèles doivent impérativement être chargés dans

l'espace mémoire contigu par ordre décroissant de taille de paramètres (Modèle de Langage 3B en premier, Embedding 300M en second, Reranker 270M en troisième) pour éviter tout échec d'allocation mémoire.

- **Gestionnaire Multitâche mzCache** : Implémentation d'un système de gestion de cache KV dynamique qui préserve l'état d'attention en mémoire lors des basculements d'applications, éliminant les latences de rechargement.

8. Pipeline Vocal Full-Duplex Temps Réel à Ultra-Faible Latence

L'interaction vocale conventionnelle souffre d'une fragmentation structurelle : la chaîne en cascade composée de la transcription (STT), du traitement textuel par le LLM et de la synthèse vocale (TTS) cumule une latence incompressible de 1 500 à 3 000 ms, interdisant toute fluidité conversationnelle.
Pour atteindre un niveau de rupture, l'assistant implémente une architecture unifiée de parole à parole (*speech-to-speech*) native, inspirée du modèle open-source **Moshi** développé par Kyutai.
Le flux de traitement s'exécute selon les étapes acoustiques suivantes :

- **Capture Microphone et Encodage Neural** : Le flux vocal de l'utilisateur est capturé en direct et échantillonné par le codec audio neuronal Mimi en trames continues de 80 ms.
- **Modélisation Multimodale Directe** : Les flux d'audio discrétisés entrent dans un modèle unifié de 7 milliards de paramètres articulé en deux transformeurs couplés :
- Le **Transformeur Temporel** analyse le contexte conversationnel et la structure temporelle.
- Le **Transformeur de Profondeur** génère simultanément les vecteurs acoustiques de la réponse vocale.
- **Décodage et Synthèse Acoustique** : Le flux audio de sortie est décompressé en continu par le décodeur Mimi et rejoué immédiatement vers le haut-parleur.

$$\text{Latence Théorique} = \text{Taille de Trame}_{\text{Mimi}} (80\,\text{ms}) + \text{Délai Acoustique} (80\,\text{ms}) = 160\,\text{ms}$$
En traitant les flux de parole de l'utilisateur et de l'assistant de manière bidirectionnelle et simultanée (*full-duplex*), ce pipeline atteint une latence de bout en bout inférieure à 200 ms. Il permet à l'utilisateur d'interrompre l'assistant en plein milieu d'une phrase et autorise la modélisation directe des intonations émotionnelles et des hésitations naturelles, sans transiter par une représentation textuelle intermédiaire.

9. Modèle de Menaces, Sécurité Systémique et Régulation

Donner à une entité logicielle autonome le contrôle intégral des entrées-sorties d'un smartphone et un accès complet aux données privées ouvre des vecteurs d'attaque inédits qu'une architecture d'ingénierie rigoureuse doit neutraliser dès sa conception.

Neutralisation du « Lethal Trifecta »

En sécurité des agents autonomes, le risque critique suprême résulte de la convergence de trois éléments :
$$\text{Accès aux Données Privées} + \text{Capacité d'Action Réelle} + \text{Lecture de Flux Externes Non Fiables}$$
Lorsqu'un agent doté d'outils d'exécution manipule des données non vérifiées provenant de l'extérieur (un e-mail entrant, un message WhatsApp, une page web ou un document

partagé), un attaquant peut concevoir une attaque par **injection indirecte de prompt** . Le texte malveillant détourne le raisonnement du modèle pour lui ordonner, par exemple, d'exfiltrer les contacts ou de transférer des fonds vers un compte tiers.
Pour neutraliser cette vulnérabilité, l'architecture applique une politique stricte de cloisonnement des contextes :

1. **Isolation des Canaux d'Entrée** : Tout contenu issu d'une application tierce ou du web est étiqueté comme donnée passive non exécutable et strictement séparé des instructions de contrôle de l'utilisateur.
2. **Garde-fous d'Autorisation avec Boucle Humaine (*Human-in-the-Loop*)** : Aucune action destructrice (suppression de fichiers, réinitialisation de paramètres), financière (virement bancaire, validation de panier d'achat) ou communicative vers l'extérieur (envoi d'e-mails ou de messages) ne peut être exécutée de manière totalement autonome. L'assistant déclenche impérativement une boîte de dialogue système protégée requérant une confirmation biométrique (empreinte digitale ou reconnaissance faciale).

Défense Contre les Injections Visuelles Adversariales (MobileInject)

Dans le benchmark MobileInject, des chercheurs ont démontré qu'un attaquant peut dissimuler des instructions textuelles malveillantes sous forme de filigranes imperceptibles ou de faibles contrastes dans des images affichées sur une page web. Lorsque le module de vision analyse l'écran, le VLM lit ces ordres invisibles à l'œil nu et dévie de sa mission.
L'assistant applique un filtre d'assainissement d'image (*Visual Sanitization*) : les zones de capture d'écran identifiées comme du contenu média externe non structuré font l'objet d'un filtrage spatial (lissage adaptatif, réduction de contraste local) afin de neutraliser les charges utiles adversariales avant leur soumission au modèle décisionnel.

Cadre Réglementaire : Le Décret Européen sur les Marchés Numériques (DMA)

Sur le plan réglementaire, l'intégration des assistants tiers bénéficie de l'intervention de la Commission Européenne. En vertu du Digital Markets Act (DMA), une décision contraignante adoptée le 16 juillet 2026 contraint Google à ouvrir 11 points d'accès système fondamentaux d'Android aux assistants d'IA concurrents d'ici le 1er août 2027 (coïncidant avec le déploiement d'Android 18).
Ces prérogatives obligatoires garantissent aux développeurs d'assistants tiers :

- L'accès direct au microphone et à la caméra avec l'écran éteint.
- La détection continue du mot-clé de réveil vocal (*wake-word*).
- L'interrogation de l'index centralisé AppSearch en arrière-plan.
- L'invocation directe des capacités applicatives via AppFunctions.
- L'accès natif aux hooks d'automatisation d'écran système (*Computer Control*).

10. Matrice de Spécification et Feuille de Route d'Implémentation

Pour concrétiser cette architecture au sein d'une application de production, les modules logiciels doivent être assemblés et validés selon une matrice de composants stricte.

Synthèse des Modules et Protocoles Techniques

Domaine Fonctionnel,Composant d'Ingénierie,Technologie / Bibliothèque Clé,Rôle Opérationnel

Exécution Système,Moteur d'Action Hybride,androidx.appfunctions / AccessibilityService,Déclenchement d'APIs natives ou injection gestuelle via dispatchGesture().
Données Locales,Passerelle Contextuelle,Android MCP / AppSearch,"Extraction locale ultra-rapide des contacts, agendas et messages sans écran."
Vision d'Écran,Détecteur & Légendeur,OmniParser v2 (YOLOv8 + Florence-2),Découpage des éléments cliquables et génération d'annotations Set-of-Mark.
Résolution Adaptative,Encodeur Multi-Régions,Pipeline Any-Resolution (Ferret-UI),Découpage dynamique des écrans étirés $9:16$ pour préserver la précision spatiale.
Contrôle Cognitif,Tri-Agent Pipeline,Mobile-Agent-v2 & Mobile-Agent-E,Séparation stricte entre Planificateur (Manager) et Décideur (Operator).
Vérification,Réflexion Multi-Niveaux,Moteur MobileUse (Action/Trajectory/Global),Détection immédiate des clics ratés et résolution des boucles d'erreurs.
Mémoire Long-Terme,Graphe & Pagination,MemGPT (RAM/Disk) & Mem0 (Graphe entités),Rétention sans dérive de contexte et diminution de 91 % de la latence $p95$.
Auto-Évolution,Moteur de Capitalisation,Algorithme Tips & Shortcuts,Extraction de leçons d'échec et accélération par macros réutilisables.
Accélération Matérielle,Inférence Embarquée,QAIRT / ONNX Runtime Mobile / CoreML,Exécution NPU optimisée en INT4/FP4 (AWQ) et ordonnancement mémoire contigu.
Interaction Vocale,Dialogue Full-Duplex,Moshi (Kyutai) avec Codec Mimi,Conversation continue parole-à-parole en temps réel avec latence < 200 ms.
Sécurité Critique,Garde-Fou Biométrique,BiometricPrompt API / Isolation Contextuelle,Validation par empreinte/FaceID sur toute action destructrice ou financière.

Protocole de Déploiement et Métriques de Validation

L'évaluation des performances du système doit être conduite sur des environnements standardisés simulant des parcs d'applications réelles :

- **Évaluation sur AndroidWorld (Google DeepMind)** :
- Mesure du taux de succès (Su) sur les 116 tâches réparties dans 20 applications réelles.
- Objectif de performance : dépasser le palier de référence des agents de première génération ($30{,}6\%$) pour atteindre plus de 65% grâce à la collaboration multi-agents et aux raccourcis d'action.
- **Évaluation sur MobileWorld** :
- Test de résistance aux consignes ambiguës, tâches de navigation multi-applications et utilisation conjointe des outils MCP.
- **Audit de Latence et Consommation Énergétique** :
- Vérification de la latence de boucle décisionnelle visuelle ($< 800\text{ ms}$ par étape sur puce de dernière génération).
- Maintien de l'empreinte thermique sous le seuil critique de $3{,}5\text{ Watts}$ en exécution continue.
- Garantie d'une latence vocale sous la barre des 200 ms pour préserver le naturel de l'échange.

En combinant une voie native privilégiée via AppFunctions et MCP avec une voie visuelle autonome hautement optimisée par Set-of-Mark et réflexion multi-échelle, cette architecture

délivre les fondations nécessaires à la création du premier assistant mobile universel, véritablement proactif et rigoureusement sécurisé.